polymarket-us · PyPI

pypi.org/project/polymarket-us

September 29, 2026



polymarket-us 2.1.0

Polymarket US Python SDK

Polymarket US Python SDK

Official Python SDK for the Polymarket US API.

Installation

```
pip install polymarket-us
```

Usage

Public Endpoints (No Authentication)

```
from polymarket_us import PolymarketUS

client = PolymarketUS()
```

```
# Get events with pagination
events = client.events.list({"limit": 10, "offset": 0, "active": True})
next_page = client.events.list({"limit": 10, "offset": 10, "active": True})

# Get a specific event
event = client.events.retrieve(123)
event_by_slug = client.events.retrieve_by_slug("super-bowl-2025")

# Get markets
markets = client.markets.list()
market = client.markets.retrieve_by_slug("btc-100k")

# Get order book
book = client.markets.book("btc-100k")

# Get best bid/offer
bbo = client.markets.bbo("btc-100k")

# Search
results = client.search.query({"query": "bitcoin"})

# Series and sports
series = client.series.list()
sports = client.sports.list()
```

Authenticated Endpoints (Trading)

```
import os
from polymarket_us import PolymarketUS

client = PolymarketUS(
    key_id=os.environ["POLYMARKET_KEY_ID"],
    secret_key=os.environ["POLYMARKET_SECRET_KEY"],
)

# Create an order
order = client.orders.create(
    {
        "marketSlug": "btc-100k-2025",
        "intent": "ORDER_INTENT_BUY_LONG",
        "type": "ORDER_TYPE_LIMIT",
        "price": {"value": "0.55", "currency": "USD"},
        "quantity": 100,
        "tif": "TIME_IN_FORCE_GOOD_TILL_CANCEL",
    }
)

# Get open orders
```

```
open_orders = client.orders.list()

# Cancel an order
client.orders.cancel(order["id"], {"marketSlug": "btc-100k-2025"})

# Cancel all orders
client.orders.cancel_all()

# Get positions
positions = client.portfolio.positions()

# Get activity history
activities = client.portfolio.activities()

# Get account balances
balances = client.account.balances()

client.close()
```

Async Usage

```
import asyncio
import os
from polymarket_us import AsyncPolymarketUS


async def main():
    async with AsyncPolymarketUS(
        key_id=os.environ["POLYMARKET_KEY_ID"],
        secret_key=os.environ["POLYMARKET_SECRET_KEY"],
    ) as client:
        # Concurrent requests
        events, markets = await asyncio.gather(
            client.events.list({"limit": 10}),
            client.markets.list({"limit": 10}),
        )
        print(f"Found {len(events['events'])} events")
        print(f"Found {len(markets['markets'])} markets")


asyncio.run(main())
```

Authentication

Polymarket US uses Ed25519 signature authentication. Generate API keys at polymarket.us/developer.

The SDK automatically signs requests with your credentials:

```
client = PolymarketUS(
    key_id="your-api-key-id",  # UUID
    secret_key="your-secret-key",  # Base64-encoded Ed25519 private key
)
```

Error Handling

```
from polymarket_us import (
    PolymarketUS,
    APIConnectionError,
    APITimeoutError,
    AuthenticationError,
    BadRequestError,
    NotFoundError,
    RateLimitError,
)

try:
    client.orders.create({...})
except AuthenticationError as e:
    print(f"Invalid credentials: {e.message}")
except BadRequestError as e:
    print(f"Invalid order parameters: {e.message}")
except RateLimitError as e:
    print(f"Rate limit exceeded: {e.message}")
except NotFoundError as e:
    print(f"Resource not found: {e.message}")
except APITimeoutError:
    print("Request timed out")
except APIConnectionError as e:
    print(f"Connection error: {e.message}")
```

Configuration

```
client = PolymarketUS(
    key_id="your-key-id",
    secret_key="your-secret-key",
    timeout=30.0,  # Request timeout in seconds (default: 30.0)
    max_retries=2,  # Automatic retries for idempotent requests (default: 2)
)
```

Retries & reliability

Idempotent requests (`GET`, `DELETE`) are retried automatically on transient failures — connection errors, timeouts, and `408`/`409`/`429`/`5xx` responses — using exponential backoff with

jitter. Non-idempotent requests such as order placement are **never** retried automatically, so a network blip cannot submit a duplicate order. Set `max_retries=0` to disable retries.

Every request sends a `User-Agent` and a generated `poly-correlation-id` so failures can be traced. The correlation id is attached to raised errors:

```
from polymarket_us import APIError

try:
    client.account.balances()
except APIError as e:
    print(e.status_code, e.message, e.request_id)
```

WebSocket (Real-Time Data)

> **Note**: WebSocket connections are async-only due to their event-driven nature. Use `asyncio.run()` when working with the sync client, or use `AsyncPolymarketUS` directly.

`SUBSCRIPTION_TYPE_ORDER` streams updates only. Request a one-shot order snapshot separately with `SUBSCRIPTION_TYPE_ORDER_SNAPSHOT` and a distinct request ID. A successful snapshot ends with an `eof: true` frame; failures use the `error` handler.

```
import asyncio
import os
from polymarket_us import PolymarketUS


async def main():
    client = PolymarketUS(
        key_id=os.environ["POLYMARKET_KEY_ID"],
        secret_key=os.environ["POLYMARKET_SECRET_KEY"],
    )

    # Private WebSocket (orders, positions, balances)
    private_ws = client.ws.private()

    def on_order_snapshot(data):
        snapshot = data["orderSubscriptionSnapshot"]
        print(f"Order snapshot: {snapshot['orders']}, eof={snapshot['eof']}")

    def on_order_update(data):
        print(f"Order execution: {data['orderSubscriptionUpdate']['execution']}")

    private_ws.on("order_snapshot", on_order_snapshot)
    private_ws.on("order_update", on_order_update)
    private_ws.on("error", lambda e: print(f"Error: {e}"))
```

```
    await private_ws.connect()
    await private_ws.subscribe("order-sub-1", "SUBSCRIPTION_TYPE_ORDER")
    await private_ws.subscribe("order-snapshot-1", "SUBSCRIPTION_TYPE_ORDER_SNAPSHOT")
    await private_ws.subscribe("pos-sub-1", "SUBSCRIPTION_TYPE_POSITION")
    await private_ws.subscribe("balance-sub-1", "SUBSCRIPTION_TYPE_ACCOUNT_BALANCE")

    # Markets WebSocket (order book, trades)
    markets_ws = client.ws.markets()

    markets_ws.on("market_data", lambda d: print(f"Book: {d['marketData']}"))
    markets_ws.on("trade", lambda d: print(f"Trade: {d['trade']}"))

    await markets_ws.connect()
    await markets_ws.subscribe("md-sub-1", "SUBSCRIPTION_TYPE_MARKET_DATA", ["btc-
100k-2025"])
    await markets_ws.subscribe("trade-sub-1", "SUBSCRIPTION_TYPE_TRADE", ["btc-100k-
2025"])

    # Keep running
    await asyncio.sleep(60)

    await private_ws.close()
    await markets_ws.close()


asyncio.run(main())
```

API Reference

Events

Method	Description
`events.list(params?)`	List events with filtering
`events.retrieve(id)`	Get event by ID
`events.retrieve_by_slug(slug)`	Get event by slug

Markets

Method	Description
`markets.list(params?)`	List markets with filtering

Method	Description
`markets.retrieve(id)`	Get market by ID
`markets.retrieve_by_slug(slug)`	Get market by slug
`markets.book(slug)`	Get order book
`markets.bbo(slug)`	Get best bid/offer
`markets.settlement(slug)`	Get settlement price

Market response type migration

The response types now match the existing JSON returned by both sync and async clients; runtime responses are unchanged. Typed callers should read book and BBO data through `marketData`. Settlement uses `slug` and a numeric `settlement`, replacing the previous `marketSlug`, `settlementPrice`, and `settledAt` declarations.

```
book = client.markets.book("btc-100k")["marketData"]
bbo = client.markets.bbo("btc-100k")["marketData"]
settlement = client.markets.settlement("btc-100k")
slug = settlement["slug"]
settlement_price = settlement["settlement"]
```

With `AsyncPolymarketUS`, await each method call before reading these keys. Handle `None` for book `stats` and `transactTime`, and BBO `bestBid`, `bestAsk`, and `lastTradePx`. Books also support `MARKET_STATE_CLOSED`.

Orders (Authenticated)

Method	Description
`orders.create(params)`	Create a new order
`orders.list(params?)`	Get open orders
`orders.retrieve(order_id)`	Get order by ID
`orders.cancel(order_id, params)`	Cancel an order
`orders.modify(order_id, params)`	Modify an order

Method	Description
`orders.cancel_all(params?)`	Cancel all open orders
`orders.preview(params)`	Preview an order
`orders.close_position(params)`	Close a position

Portfolio (Authenticated)

Method	Description
`portfolio.positions(params?)`	Get trading positions
`portfolio.activities(params?)`	Get activity history

Account (Authenticated)

Method	Description
`account.balances()`	Get account balances

Series

Method	Description
`series.list(params?)`	List series
`series.retrieve(id)`	Get series by ID

Sports

Method	Description
`sports.list()`	List sports
`sports.teams(params?)`	Get teams for provider

Search

Method	Description
`search.query(params?)`	Search events (includes nested markets)

WebSocket (Authenticated, Async-Only)

Method	Description
`ws.private()`	Create private WebSocket connection
`ws.markets()`	Create markets WebSocket connection

WebSocket methods (`connect()`, `subscribe()`, `close()`) are async and must be awaited.

Private WebSocket Events:

- `order_snapshot` - Initial orders snapshot
- `order_update` - Order execution updates
- `position_snapshot` - Legacy snapshot event; the current gateway sends no position snapshot
- `position_update` - Position changes
- `account_balance_snapshot` - Initial balance
- `account_balance_update` - Balance changes
- `rfq_event` - RFQ and quote lifecycle events
- `heartbeat` - Connection keepalive
- `error` - Error events
- `close` - Connection closed

`position_update` now also recognizes `positionSubscription`, and `account_balance_update` recognizes `accountBalancesUpdate`. Existing dispatch aliases are retained, and both the named callback and `message` receive the original envelope without renaming fields. An empty `error` string no longer suppresses a successful data callback.

RFQ subscriptions deliver lifecycle events through `rfq_event` without an initial snapshot. Market filters are not supported.

```
from polymarket_us.websocket import RFQEvent


def on_rfq_event(data: RFQEvent) -> None:
```

```
    event = data["rfqEvent"]
    created = event.get("rfqCreated")
    rfq = created["rfq"] if created is not None else None
    if rfq is not None:
        print(rfq["id"], rfq.get("qtyDecimal"))


private_ws.on("rfq_event", on_rfq_event)
await private_ws.subscribe_rfq("rfqs-1")
```

Other event keys are `rfqClosed`, `quoteCreated`, `quoteDeleted`, `quoteAccepted`, `quoteConfirmed`, and `quoteExecuted`. Timestamps and nested RFQ/quote objects may be null. Portfolio activity trades also expose `qtyDecimal` as an exact decimal string; use it instead of the rounded `qty` when fractional quantities matter.

Private callback type migration (2.0.0)

`PositionUpdate`, `AccountBalanceSnapshot` and `AccountBalanceUpdate` now describe the current gateway payloads. Replace `positionSubscriptionUpdate.position` with `positionSubscription.beforePosition` / `afterPosition`. Replace the flat `accountBalanceSubscriptionSnapshot` and `accountBalanceSubscriptionUpdate` fields with `accountBalancesSnapshot.balances` and `accountBalancesUpdate.balanceChange.beforeBalance` / `afterBalance`. Before/after values and timestamps can be `None`; balance entries use `currentBalance` and `buyingPower`. Use `netPositionDecimal` and the other decimal quantity fields for exact fractional positions; the older quantity fields are rounded. Position types include nullable cost fields and combo leg details. Balance reservation and display fields are optional: an absent value is unknown, while `0` is a known zero.

```
from polymarket_us.websocket import AccountBalanceSnapshot, AccountBalanceUpdate,
PositionUpdate


def on_position(data: PositionUpdate) -> None:
    change = data["positionSubscription"]
    print(change["beforePosition"], change["afterPosition"])


def on_balances(data: AccountBalanceSnapshot) -> None:
    print(data["accountBalancesSnapshot"]["balances"])


def on_balance(data: AccountBalanceUpdate) -> None:
    after = data["accountBalancesUpdate"]["balanceChange"]["afterBalance"]
    if after is not None:
        print(after.get("currentBalance"), after.get("currency"))
```

```
private_ws.on("position_update", on_position)
private_ws.on("account_balance_snapshot", on_balances)
private_ws.on("account_balance_update", on_balance)
```

These annotations describe current server messages. Applications consuming legacy aliases must continue reading their original payload shapes. `PositionSnapshot` remains exported for legacy messages; use `portfolio.positions()` for an initial positions read. Position subscriptions deliver subsequent changes only.

Markets WebSocket Events:

- `market_data` - Full order book updates
- `market_data_lite` - Lightweight price data
- `trade` - Trade notifications
- `heartbeat` - Connection keepalive
- `error` - Error events
- `close` - Connection closed

Requirements

Python 3.10+

Development

```
# Install dev dependencies
pip install -e ".[dev]"

# Run tests
pytest

# Run linting
ruff check .

# Run type checking
mypy polymarket_us
```

License

MIT

Project links

Key dates

▶ PyPI data

1 maintainer

Credits

License expression

Requires

Provides Extra

Tags

Classifiers



Vercel is a Maintaining sponsor of the Python Software Foundation.